



A collaborative scheduling method for cloud computing heterogeneous workflows based on deep reinforcement learning

Genxin Chen^a, Jin Qi^b, Ying Sun^c, Xiaoxuan Hu^b, Zhenjiang Dong^c, Yanfei Sun^{b,*}

^a School of Automation, Nanjing University of Posts and Telecommunications, Nanjing, Jiangsu Province, China

^b School of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing, Jiangsu Province, China

^c School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, Jiangsu Province, China

ARTICLE INFO

Article history:

Received 16 July 2022

Received in revised form 23 November 2022

Accepted 26 November 2022

Available online 29 November 2022

Keywords:

Heterogeneous workflows
Cloud computing
Deep reinforcement learning
High-dimensional objectives
Collaborative scheduling
Adaptive mechanism

ABSTRACT

Aiming at the problem of low overall service quality caused by the disordered collaboration of heterogeneous workflows and discontinuous task execution in cloud computing scenarios, this paper proposes a collaborative scheduling method for heterogeneous workflows in cloud computing based on deep reinforcement learning. The method optimizes workflow makespan, cost, fairness and continuity in cloud computing under the constraints of task execution continuity. First, the structure and time sequence features are extracted for the dynamic scheduling process, and a reasonable scheduling decision support feature set is constructed. Second, a time-step adaptive scheduling mechanism is designed to simplify redundant information in the scheduling process and enables the agent to achieve efficient learning. In addition, using equilibrium, priority and preference scheduling strategies, an immediate-lag compound reward mechanism and a scheduling-switching hybrid action are designed to achieve a unification of the agent's learning objectives and actual scheduling requirements. Finally, by constructing a simulation platform and conducting comparative experiments with four other algorithms, the results show that the proposed method has advantages in collaborative optimization of high-dimensional objectives under task continuity constraints. Including the task loading strategy can optimize the makespan performance by 16.6% and improve the fairness index by 5.3%.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

With the rapid development of technologies such as cloud computing, virtualization, and containers, microservice applications in the form of workflows are gradually increasing in cloud computing scenarios, and the complexity is rapidly increasing. Meeting the diversity of computing business demands under an elastic cloud architecture is an urgent issue that needs to be considered [1]. As an ordered collection of massive computing tasks, workflows usually appear in the form of directed acyclic graphs [2] and are widely executed with the growth of complex computing requirements in various application fields. The workflow usually contains a large number of task nodes, and the concurrent computing demand fluctuates greatly, especially in heterogeneous workflow combinations. It is more difficult to determine the demand law. Although the large-scale computing resources in the cloud computing environment help to ensure the stability of computing services [3], it is still necessary to solve

the problem of optimal adaptation between computing services amid the intensive utilization of resources and the computing requirements of heterogeneous workflows.

As an NP-complete problem, the workflow scheduling problem is difficult to solve [4]. The existing methods have conflicts among multiple objective optimizations, which restricts the scheduling effect of workflows. In real computing scenarios, computing services often have various forms [5]. The workflow of similar services will lead to changes in attributes such as calculation amount and transmission amount under the change of demand, resulting in uncertainty of task completion time [6–8]. Various services may also have great differences in structure, such as depth, width, and computational complexity [9]. Most of the existing work focuses on the scheduling optimization of a single workflow, and it is difficult to solve the optimization problem of collaborative scheduling for multiple workflows in cloud computing scenarios. The emergence of heterogeneous workflows enhances the resource demand differences in the computing process, deteriorates the balance of supply and demand, and causes resource waste or service quality degradation. Due to the superposition of workflow traffic and a mixture of various structures and characteristics, the characteristics of specific workflows are often difficult to reflect, resulting in the dynamic nature of the

* Corresponding author.

E-mail addresses: genxinchen@163.com (G. Chen), qijin@njupt.edu.cn (J. Qi), sunyingnupt@163.com (Y. Sun), xiaoxuanhu@njupt.edu.cn (X. Hu), dongzhenjiang@njupt.edu.cn (Z. Dong), sunyanfei@njupt.edu.cn (Y. Sun).

computing environment [10]. Some static scheduling methods have achieved good results in experiments [11], such as designing a rescheduling mechanism to deal with the failure of task execution [12–15]. However, due to the lack of consideration of the multidimensional dynamic characteristics of the computing environment, the aforementioned methods are difficult to adapt to practical needs. The way that cloud computing service providers effectively match appropriate resources to dynamic demands is the key to cloud computing service quality and cost optimization [16]. Per the requirement of concurrent dynamic scheduling of multiple groups of heterogeneous workflows, effectively extracting and utilizing the key characteristics of the cloud computing environment and designing a scheduling mechanism to control the trend of nonequilibrium matching in time is a problem that must be considered in optimizing the cloud computing environment.

Traditional workflow scheduling optimization mostly takes cost and efficiency as the main objectives [17]. However, computing services in real scenarios often have multiple requirements, such as execution fairness and continuity between tasks [18,19]. Fairness represents the balance of resource acquisition for different computing tasks, and is the embodiment of the fair service order of cloud computing. Continuity represents the coherence between services or tasks and is a problem that must be considered in real-time services under fine-grained task promotion. Various complex requirements increase the dimensions of the objectives and constraints of workflow scheduling optimization problems, and optimizing some of the conflicting objectives is the mainstream research work [20–22]. However, applications in real scenarios often have high-dimensional objectives, and it is difficult to achieve the ideal state of practical applications by only optimizing resources on the supply side or optimizing service quality on the demand side [23–25]. With the increasing complexity of business scenarios, it is necessary to study the collaborative method of optimizing high-dimensional goals between the supply and demand in workflow scheduling to achieve a good balance between conflicting goals and nonconflicting goals at the Pareto frontier.

This paper proposes a scheduling method for heterogeneous workflows in cloud computing scenarios named the heterogeneous workflow collaborative scheduling method based on deep reinforcement learning (HCDRL). Under the constraint of service continuity, we optimize the execution makespan, cost, fairness and continuity of heterogeneous workflows in cloud computing. By constructing a state space oriented to heterogeneous workflow scheduling, deep reinforcement learning is introduced into the heterogeneous workflow scheduling process, corresponding mechanisms and strategies are designed, and compound rewards are used to optimize the representation of high-dimensional objectives, thus realizing the collaborative scheduling of heterogeneous workflow high-dimensional objectives. The main contributions of this paper are as follows:

(1) For the complex environment of concurrent scheduling of heterogeneous workflows, a scheduling model based on multi-workflow collaboration is constructed. The deep reinforcement learning method based on the D3QN algorithm is introduced to solve the problem, and the high-dimensional optimization of the collaborative scheduling process of heterogeneous workflows under continuity constraints is realized.

(2) A multilayer task state awareness algorithm for heterogeneous workflows is proposed to accurately observe the state of workflow time series tasks. In this paper, we discuss designing a time-step adaptive mechanism to reduce the data dimension. High-dimensional strategies such as equilibrium, priority, and preference are adopted to realize the integration of prior knowledge and artificial intelligence.

(3) The cloud server state control and task scheduling decisions are integrated to realize the follow-up optimization of cloud computing resources. The construction of a compound reward mechanism unifies the multiscale reward differences, and achieves a stable increase in the value of intelligent strategies by reasonably representing the optimization objectives.

The structure of the follow-up content of this paper is as follows. Section 2 summarizes the related research work of this paper. Section 3 describes the heterogeneous workflow scheduling model, scheme and mechanism. Section 4 presents the scheduling strategy and algorithm proposed in this paper, including the design of deep reinforcement learning-related content. Section 5 analyzes and compares the effects of the proposed algorithm in this paper. Section 6 summarizes the work of the full text and presents the prospect of future work.

2. Related work

With the popularization of artificial intelligence and other applications, cloud computing scenarios carry a large number of real computing services. Optimizing the supply side and the demand side to meet the actual needs flexibly will be directly linked to the quality of services related to real scenarios. As a typical task representation form, the workflow structure is widely used in various resource scheduling and service matching scenarios.

2.1. Workflow scheduling scenarios

Some studies have proposed a communication bottleneck solution based on traffic overhead optimization for container-based large-scale scientific workflow scheduling [26]. Through the awareness of data location in the workflow, an agent negotiation mechanism and a dynamic workflow reorganization strategy are designed, and task replication is used to optimize the execution time and traffic overhead of the workflow. In the flexible manufacturing production environment, there is also research on scheduling production resources based on multi-agent reinforcement learning methods [27]. By modeling the factory production line topology and product flow, a method from single-agent optimization to multiagent collaboration is constructed to stably handle global optimization scheduling in a dynamic production environment. In workflow scheduling in fat tree networks, research on the energy consumption caused by network resource transmission between computing nodes is also of great importance [28]. Through the network workflow scheduling method based on a heuristic list, including top-level task calculation, task subterm initialization and other processes, the energy consumption of workflow scheduling is reduced under the task execution time constraints.

The scheduling problem of workflows in various scenarios has commonalities and differences in task graph structure and the actual requirements [29]. The basic model has high similarity, but there are large differences in specific optimization. Therefore, it is necessary to study dynamic scheduling methods that can adapt to dynamic scenarios and heterogeneous multiple workflows.

2.2. Workflow scheduling optimization objectives and constraints

Diverse scenarios form different objectives in the workflow scheduling process, such as time and cost, and constraints are mostly deadlines and energy consumption. For the virtual machine cost optimization problem under the workflow deadline constraint, the elastic scheduling of microservices that combines task scheduling and automatic scaling is a feasible idea [1]. The urgency-based workflow scheduling algorithm is used to assign

tasks, while determining the type and number of cloud computing instances currently needed. An improved HEFT algorithm based on greedy resource supply is used to optimize workflow execution time under cost constraints [30]. Deng et al. proposed a workflow multilevel graph partitioning algorithm, which divides workflow tasks into multiple parts for execution [31]. Under the constraints of load balancing and data volume, the delay caused by cross-data center transmission during workflow execution is optimized. Cadorel et al. established an optimal solution model for workflow scheduling energy consumption and user fairness and designed an online scheduling algorithm that is sensitive to task time limits [10]. Dickinson et al. established a joint resource allocation framework in a multicloud environment to meet the computing security and performance requirements of data-intensive businesses [32]. By defining the security specifications in the workflow execution process, alignment technology is used to homogenize various domain resource policies for resource domain and network path selection. Zhu et al. used the membership function to link the fuzzy membership of the multiobjective optimization problem in the workflow scheduling process and proposed an optimal foraging scheduling algorithm based on fuzzy relative entropy [33].

With the increasing complexity of application scenarios and businesses, the dimensions of objectives and constraints gradually increase, which leads to the need for high-dimensional objective optimization and scheduling [34,35].

2.3. Workflow scheduling optimization solution

For the workflow scheduling problem solution, the existing work usually adopts analytical, heuristic, artificial intelligence and other methods combined with knowledge of fuzzy theory and game theory. The author in [6] proposed an adaptive ant colony optimization algorithm, which reduces the search space of the algorithm by designing an adaptive solution construction method. The weight adaptive mechanism related to the deadline is designed to optimize the execution time of the workflow under the condition of meeting the requirements of the deadline. Hu et al. proposed dynamic voltage–frequency scaling and relaxation time recovery algorithms to solve the energy optimization problem of massively parallel workflows under time and reliability constraints in heterogeneous distributed computing platforms [26]. Li et al. proposed a nondominated sorting genetic algorithm based on a scheduling score and workflow dynamic hierarchy, which jumped out of the local optimum by adopting an adaptive adjustment strategy [25]. The authors in [36] proposed a list scheduling algorithm that assigns priority to tasks and reduces the amount of scheduling through workflow task clustering to optimize scheduling overhead and execution time. Ran et al. proposed a task static scheduling method based on deep reinforcement learning for the optimization problem of workflow task response time in the dynamic heterogeneous environment of cloud computing, and established a task average response time optimization solution model under load balancing constraints [37].

In the current situation of the increasingly complex workflow business, the dynamic scheduling method of high-dimensional objects for heterogeneous workflows is one of the future research directions. Furthermore, artificial intelligence methods can better solve the problem, especially in the case of poor applicability of analytical methods and low sample utilization of heuristic methods.

3. Model building and problem description

3.1. Multiworkflow collaborative scheduling model

Given object labels i, j, e and f , workflow W_i is usually described in the form of a directed acyclic graph, and the nodes of the directed acyclic graph represent task t_{ij} in workflow W_i . The edges between the nodes represent the relationship $R(t_{im}, t_{jn})$, ($0 < m, n < j$) between the tasks. The process of workflow scheduling is the bottom-up analysis process of directed acyclic graphs following relational constraints. In a multicloud environment, cloud computing services are usually composed of multilayer structures, such as server clusters, C_i , hosts, $host_{ij}$, and virtual machines, V_{ijk} . Multiworkflow coscheduling is the process of assigning an appropriate V_{ijk} to multiple t_{ij} . This paper proposes a multiworkflow collaborative scheduling model, as shown in Fig. 1. The left side of the model is the heterogeneous workflow analysis module, the right side is the cloud server state control module, and the middle is the multiworkflow collaborative scheduling module based on deep reinforcement learning. This paper matches the tasks in the unconstrained task pool with the virtual machine resources in the available virtual machine pool by continuously observing the multiscale state of the workflow and the server. At the same time, the state of the cloud computing server is controlled according to the dynamic load status of the server to optimize the high-dimensional objective proposed in this paper.

3.1.1. Analysis of heterogeneous workflow

Multiple dynamic heterogeneous workflows are aggregated in a workflow pool $WP = \{W_0, W_1, \dots, W_i\}$, where $W_i = \{t_{i0}, t_{i1}, \dots, t_{ij}\}$ is a collection of tasks in the workflow. $F(W_i) = \{t_{i0}, t_{i1}, \dots, t_{if}\}$ represents the initial task set of W_i . $E(W_i) = \{t_{ie}, t_{i1}, \dots, t_{ie}\}$ represents the end task set of W_i , and the total number of workflows in a certain period is set to M . The total number of tasks is $All(t_{ij})$, and the total task calculation amount is $ACp(t_{ij})$. Moreover, the tasks in the initial task set contain subtasks $S(t_{ij})$, and the tasks in the end task set contain parent tasks $P(t_{ij})$. Tasks other than the initial and end task sets contain both parent and child tasks. The workflow scheduling process follows the parent–child relationship; that is, when all the parent tasks of the child task are completed, the child task can be scheduled.

The tasks in the workflow have various attributes, such as the attribute of the calculation amount $Cp(t_{ij})$ of the task node, which is the fundamental basis for scheduling. The number of parent tasks $AP(t_{ij})$ and the number of child tasks $AS(t_{ij})$ of the task node affect the task sequence concurrency in the workflow scheduling process. The parent task affects the execution timing of subsequent tasks, and the child task affects the number of subsequent tasks that can be executed at the same time. In addition, the number of child task series $AS_{root}(t_{ij})$ is also an important indicator for decision-making of the current task, which represents the multigeneration child task set of the current task and affects the timing concurrency of the long-term scheduling process. The moment $Start_i$ when the workflow enters the task sequence and the scheduling order determine the workflow scheduling waiting time $Wait(W_i)$. Similarly, there is the task waiting time $Wait(t_{ij})$ after the workflow starts to execute. The former describes the waiting time to start execution after the workflow arrives, and the latter describes the waiting time for a specific task to start execution after the workflow starts executing. In addition, the model proposes the waiting time $FrWt(t_{ij})$ after the task is unconstrained to describe the continuity of computing services:

$$FrWt(t_{ij}) = TIME - \max(Wait(P(t_{ij}))) \quad (1)$$

intervals, and the task computation cost is defined as follows:

$$f_2(x) = \sum_{i=0}^c \sum_{j=0}^L \text{host_stat}_i * \text{time}_{\text{step}_j} \quad (5)$$

where c is the total number of hosts in the computing environment, and host_stat_i is the cost coefficient of the host in the current state. The fairness of the computing services is defined as follows:

$$f_3(x) = \sqrt{\frac{\sum_{i=1}^M (\sum_{j=0}^L W_i(\text{time}_{\text{step}_j}) - m_t)^2}{M}} \quad (6)$$

$f_3(x)$ is the standard deviation of the sum of the computing service resources $W_i(\text{time}_{\text{step}_j})$ obtained by each workflow in the scheduling process, and m_t is the mean of $W_i(\text{time}_{\text{step}_j})$. The task continuity objective is defined as follows:

$$f_4(x) = \frac{\sum_{i=0}^M \sum_{j=0}^{\text{Fct}(t_{ij})} \text{FrWt}(t_{ij})}{\text{Fct}(t_{ij})} \quad (7)$$

It can be seen that the smaller the makespan, cost, fairness and continuity indicators, the better, and the larger the continuity constraint compliance value, the better. However, the makespan, computing continuity and computing cost are conflicting indicators, and the two should achieve the optimal cost-to-makespan and continuous ratio on the basis of satisfying constraints. Under the premise of limited resources, the fairness and continuity of computing services are also conflicting objectives. When computing resources are evenly allocated to each workflow, the computing resources obtained by some workflows may not be able to meet the computing requirements of multiple subtasks. In this case, the waiting time after the task is free is too long, resulting in a violation of the task's continuity constraint. However, when computing resources focus on some workflows, the waiting time of other workflows will be too long, which violates the law of the computing service value and is not conducive to the establishment of a fair computing service order.

4. Heterogeneous workflow collaborative scheduling

To address the above problems, this section proposes a heterogeneous workflow collaborative scheduling method based on deep reinforcement learning. In this section, the basic scheduling framework is constructed to support the interaction process between the scheduling environment and the agent, as shown in Fig. 2. The basic elements of the reinforcement learning algorithm are state, action and reward. Therefore, this paper discusses designing the mechanism and algorithm related to the heterogeneous workflow scheduling problem according to the three elements. State is the basic input of an agent. We describe the state awareness algorithm in Section 4.1 to extract key features from the scheduling environment and provide basic support for decision-making. On this basis, we discuss designing the state change mechanism in Section 4.2, that is, the time-step adaptive state observation and decision-making under dynamic changes. Section 4.3 describes the action mechanism of the agent. We implement a server-oriented preference strategy and a task-oriented balance and priority strategy through high-dimensional action integration, and realize the divide-and-conquer of workflow task loading and task scheduling. Finally, in Section 4.4, we describe designing the compound rewards linked with the objective, integrate the state, action and reward, and use the D3QN algorithm to optimize and solve the heterogeneous workflow scheduling problem.

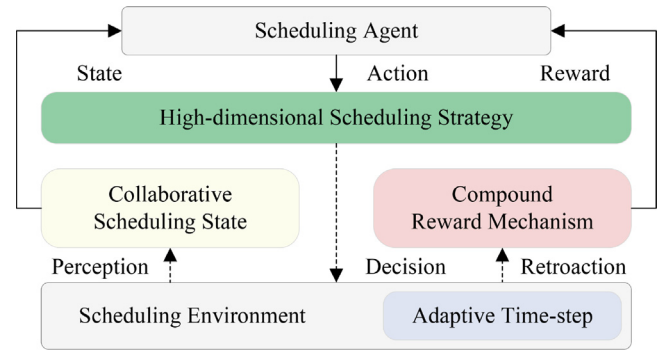


Fig. 2. Heterogeneous workflow collaborative scheduling framework.

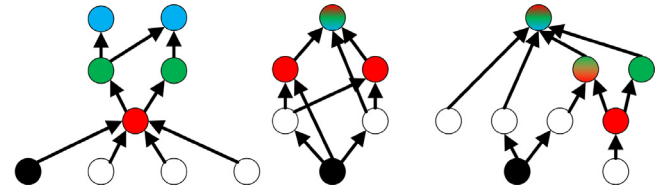


Fig. 3. Heterogeneous workflow multilayer task view.

4.1. Collaborative scheduling state awareness algorithm

In the process of workflow scheduling, tasks have three states: unscheduled, scheduled but not completed, and completed. The scheduler needs to schedule unscheduled tasks to the virtual machine for execution and update the workflow and cloud server status in real-time. It can be seen in Fig. 1 that the key to multiworkflow collaborative scheduling lies in the high-dimensional information observation space, including workflow features, task features, host features, virtual machine features, etc. The observation of workflow features is complicated and involves multilayer state sensing of heterogeneous workflows. Observing the multilayer task state of a workflow uniformly and efficiently is an important problem to be solved in this paper. To obtain the quantity characteristic L_i of multilayer child tasks, the characteristic of calculation amount Cp_i , and the characteristic of waiting time FrW_i when multiple workflows are scheduled, this paper proposes a multilayer task state awareness algorithm for a workflow, as shown in Algorithm 1.

The algorithm obtains the multilayer features of the workflow by traversing layer-by-layer. The multilayer child task quantity feature L_i represents the number of hierarchical tasks in the multilayer task view of the heterogeneous workflow, the calculation amount feature $Cimp_i$ represents the average calculation amount in the corresponding view, and the waiting time feature FrW_i represents the average waiting time after the parent task starts executing in the corresponding view. Multilayer features can better focus on the timing characteristics of heterogeneous workflows under the parent-child constraints. As shown in Fig. 3, black task nodes represent completed tasks, and white task nodes represent freely schedulable tasks. Red, green, and blue nodes represent the next, second, and third layers of all free tasks. The mixed color nodes indicate that it belongs to multiple layers at the same time. For this type of multiparent node, in Algorithm 1, Line 13 only calculates the weights of a single-parent series in the corresponding hierarchy. Obviously, the characteristics of different levels of heterogeneous workflows vary greatly under different execution degrees, which can provide rich decision support.

Algorithm 1: Multilayer task state wareness algorithm

Input: The currently free unscheduled task set *Freet*, the currently uncompleted workflow full task set *Fullt*, number of perception layers *N*

Output: Multilayer child task quantity characteristic L_i , calculation quantity characteristic Cp_i , free waiting time characteristic $FreW_i$

```

1 childset = {},  $L_0 = 0$ ,  $Cp_0 = 0$ ,  $FreW_0 = 0$ 
2 while  $k < N$  do
3   for  $i \in Freet$  do
4     for  $j \in Fullt$  do
5       if  $j \in S(i)$  then
6          $r = 0$ 
7         for  $m \in P(j)$  do
8           if  $j \notin childset \ \& \ m == i$  then
9              $childset \leftarrow j$ 
10            else if  $m \in Freet \ \& \ m! = i$  then
11               $r = r + 1$ 
12            end
13             $L_k = L_k + 1 / (AP(j) - r)$ 
14          end
15        end
16      end
17    end
18    for  $i \in childset$  do
19      compute  $Cp_k$ , compute  $FreW_k$ 
20    end
21     $Cp_k = Cp_k / L_k$ ,  $FreW_k = FreW_k / L_k$ 
22     $k = k + 1$ ,  $Freet = childset$ ,  $childset = \{\}$ 
23  end

```

The three sets of features combined with the basic features of the workflow and the host constitute the information observation space of collaborative scheduling. Therefore, in the real-time scheduling process under the multiworkflow parent-child constraints, the multilayer concurrent computing requirement information of the workflow can be represented more directly and comprehensively.

4.2. Time-step adaptive scheduling mechanism

Workflow scheduling is a dynamic process, and the number of states generated can grow infinitely with time subdivision. A large amount of useable data is not conducive to neural network fitting. Therefore, reasonable control of sample dimensions is the key to applying artificial intelligence methods. The scheduling process of the workflow involves multiple task assignments, so multiple actions need to be performed within the workflow completion time. In the case of only considering task scheduling, the number of scheduling actions is only related to the number of tasks. However, if the state control and other actions of the server are considered, the number of scheduling actions is related to the fine-grained time division. In the dynamic scheduling process, the scheduling strategy can occur at any time, and the fine-grained time division will determine the degrees of freedom of the task scheduling strategy. However, the number of state observations is also related to the fine-grained time division. The higher the fine-grainedness is, the more comprehensive and accurate the understanding of the environment is; when the fine-grainedness is lower, the major changes may be ignored. However, the higher the fine-grained time division is, the more scheduling actions are generated in the scheduling time interval, resulting in an increase in time series observation features, which is not conducive to the

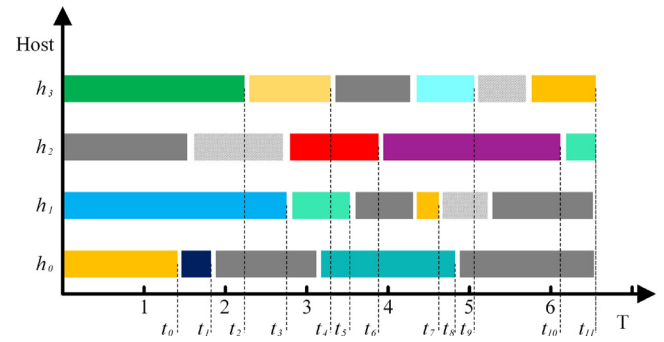


Fig. 4. Time-step adaptive scheduling mechanism.

application of deep reinforcement learning algorithms. Therefore, it is necessary to reduce the time fine-grainedness on the premise of ensuring the degree of freedom of the scheduling policy and the richness of information.

To reduce the fine-grainedness of time, the method of increasing the time division value is usually adopted. By increasing the increment value of the scheduling time, the counts of time steps in the scheduling time interval of the workflow task are reduced, and the cumulatively executed scheduling actions are also reduced accordingly. However, this method will bring about a delay in decision-making. The completion time of the task is not consistent with the time-step, so after the task is completed, it is necessary to wait for the end of the time-step to make a task scheduling decision. As shown in Fig. 4, host h_0 has completed the computing task at time t_1 , but it needs to wait until the time-step is completed before executing the next task or switching the host state. Obviously, the waiting time will cause a huge waste of resources.

In this paper, the time-step adaptive scheduling mechanism is adopted, and the completion time of each task is used as the scheduling node. As shown in Fig. 4, the vertical axis represents different hosts, and the horizontal axis represents the status of the host, where gray means the host is sleeping, stripes mean the host is idle, and colored means the host is performing tasks. Taking the start time of the scheduling process as the first scheduling node, the time-step is updated when one of the conditions, such as the online host resources, is allocated or the current schedulable task is completely scheduled. The time-step is the interval length from the current time to the time when the latest task is completed, which is defined as follows.

$$\begin{aligned}
 time_{step} = & \min (Start_i + Wait (W_i) \\
 & + Wait (t_{ij}) + \frac{Cp (t_{ij})}{Mips (V_{ijk})} - TIME)
 \end{aligned} \quad (8)$$

Obviously, the value of $time_{step}$ depends on the number of tasks and the computing power of the host; that is, the time-step needs to adapt to the dynamic computing service environment. After updating the initial time-step, a new scheduling decision can be made and then updated again, and the cycle repeats until all tasks are completed. As shown in Fig. 4, the time-steps are set to t_0 , $t_1 - t_0$, $t_2 - t_1$, etc. In this case, the decisions in the scheduling process are limited to the moment after each time-step. The number of decisions at this moment is limited, which can not only ensure the learnability of the time series rules, but also improve the data value and utilization efficiency.

4.3. High-dimensional action integration scheduling strategy

In reinforcement learning, for some simple tasks, the agent's action is easier to design, such as the inverted pendulum problem,

which only needs to design left and right actions. For complex problems, the real action dimension is too large, making reinforcement learning extremely difficult. Therefore, designing high-dimensional strategies to integrate actions for specific problems combined with domain knowledge is a feasible direction for knowledge and data collaborative intelligence. In addition, undesigned actions tend to make the agent fall into an abnormal environment, such as scheduling actions when no available resources are available. Although the penalty term constraint can be set, the experience playback pool is filled with a large number of low-value state–action pairs, which also increases the learning difficulty.

As a result, this paper proposes a high-dimensional scheduling action based on a preference strategy, equilibrium strategy and priority strategy to achieve the unity of scheduling value and freedom of action. The preference strategy is mainly designed for basic actions, such as scheduling, dormancy, and activation. The scheduling preference policy is oriented to virtual machines and mainly consists of three basic preferences. By arranging and combining the three preferences of computing efficiency, computing cost, and host load rate, six virtual machine service preferences that can be selected by the current task are obtained. The function of the scheduling preference policy is to avoid scheduling tasks to hosts with no available resources, and the design of multiple preferences ensures the possibility of exploring different scheduling actions. Activation and dormancy preference policies are designed for the host. The activation preference strategy selects the dormant host that can be activated when the agent chooses to determine more available resources, including the highest efficiency preference and the lowest cost preference. The dormancy preference strategy selects the host with the lowest resource usage, which can become dormant when the agent chooses to reduce the host's standby consumption.

For the task itself, this paper also describes designing a task preloading strategy, including the equilibrium strategy and the priority strategy. Since there may be a large number of tasks waiting to be scheduled at the same time in the scheduling process, it is impossible for the scheduler to observe the detailed characteristics of each task for scheduling at the same time. It is a feasible method to optimize scheduling under concrete observation for a small number of tasks. Therefore, task preloading replaces the scheduler to observe the schedulable task set and selects the current optimal task from several key feature dimensions. The equilibrium strategy is shown in Algorithm 2.

On the basis of considering the task continuity constraints, the balanced strategy task loading algorithm first considers the fairness of workflow resource allocation (Lines 1–6), and selects the workflow task with the lowest resource acquisition rate $Acec(W_i)$ after the workflow scheduling starts. $Acec(W_i)$ is calculated as follows:

$$Acec(W_i) = \frac{\sum_{j=0}^L W_i(T_j)}{TIME - Start_i} \quad (9)$$

Second, the task waiting time is used as the scheduling basis, and tasks with long waiting times are preferentially scheduled (Lines 7 to 10). On this basis, we continue to consider the number of child tasks (Lines 11 to 24). When the task time constraint feature value of more than half of the tasks in the filtered task set exceeds half of the time constraint value, we select the task with the least number of child tasks. Otherwise, the task with the largest number of child tasks is selected to balance the degree of scheduling freedom and the compliance of task time constraints. Finally, the algorithm selects the task with the most parent task nodes in the filtered task set (Lines 25 to 29), which simplifies the task-related resource inventory in the workflow scheduling process to the greatest extent.

Algorithm 2: Equilibrium policy task loading algorithm

Input: $Acec(W_i)$, the time constraint tim_c
Output: Preload task t_{ij}

```

1  $respool = \{\}, timp = \{\}, chilp = \{\}, parp = \{\}$ 
2 for  $i < M$  do
3   | Assign  $\min(Acec(W_i))$  to  $respool$ 
4 end
5 if  $\text{len}(respool) == 1$  then
6   |  $t_{ij} = respool[0]$ 
7 else
8   | Assign  $\max(FrWt(respool))$  to  $timp$ 
9   | if  $\text{len}(timp) == 1$  then
10    |  $t_{ij} = timp[0]$ 
11  else
12    for  $i$  in  $timp$  do
13      |  $count = 0$ 
14      | if  $FrWt(i) > 0.5 * tim\_c$  then
15        |  $count++ = 1$ 
16      | end
17    end
18    if  $count > 0.5 * \text{len}(timp)$  then
19      | Assign  $\min(AS(timp))$  to  $chilp$ 
20    else
21      | Assign  $\max(AS(timp))$  to  $chilp$ 
22    end
23    if  $\text{len}(chilp) == 1$  then
24      |  $t_{ij} = chilp[0]$ 
25    else
26      | Assign  $\max(AP(chilp))$  to  $parp$ 
27      |  $t_{ij} = \text{random choice } parp$ 
28    end
29  end
30 end

```

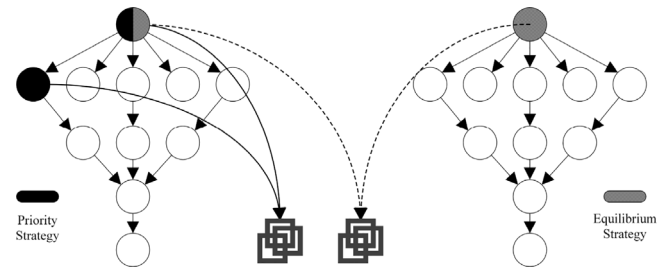


Fig. 5. Equilibrium-priority task scheduling policy.

However, if the service fairness of the workflow is pursued blindly, in the case of too many subtasks, the concurrent demand of the workflow tasks at the same time may be too large, and the task waiting time may be too long. As shown in Fig. 5, when the equilibrium strategy is selected, the two workflows will directly release a large number of schedulable tasks. In the case of limited resources, the server cannot meet the continuity constraints between tasks in time. Therefore, this paper proposes a priority policy task loading algorithm, as shown in Algorithm 3.

Lines 1 to 6 in Algorithm 3 are consistent with Algorithm 2. In Lines 7 to 10, the schedulable tasks in the workflow with the highest completion rate are selected to be loaded first. In Lines 11 to 14, the task with the most child tasks in the task set is selected to be loaded first to improve the resource occupancy rate of this workflow. In Lines 15 to 19, the task with the most parent tasks in the task set is selected to be loaded first, which simplifies

Algorithm 3: Priority policy task loading algorithm

Input: $A_{cec}(W_i)$, the time constraint tim_c
Output: Preload task t_{ij}

```

1  $respool = \{\}, workfi = \{\}, chilp = \{\}, parp = \{\}$ 
2 for  $i < M$  do
3   Assign  $\min(A_{cec}(W_i))$  to  $respool$ 
4 end
5 if  $\text{len}(respool) == 1$  then
6    $t_{ij} = respool[0]$ 
7 else
8   Assign  $\max(\text{FinRate}(respool))$  to  $workfi$ 
9   if  $\text{len}(workfi) == 1$  then
10     $t_{ij} = workfi[0]$ 
11  else
12    Assign  $\max(AS(workfi))$  to  $chilp$ 
13    if  $\text{len}(chilp) == 1$  then
14       $t_{ij} = chilp[0]$ 
15    else
16      Assign  $\max(AP(chilp))$  to  $parp$ 
17       $t_{ij} = \text{random choice } parp$ 
18    end
19  end
20 end

```

the task-related resource inventory in the workflow scheduling process to the greatest extent.

The addition of the equilibrium policy and priority policy enables the agent to master mature and feasible scheduling skills without much training and makes the agent only need to learn the use of knowledge-driven skills, which improves the learning efficiency. On this basis, to prevent the poor learning effect caused by different action categories, this paper combines scheduling preference, host activation, dormancy preference and task loading preference to form a base class action. Before each scheduled action is executed, host activation, dormancy action or no action is performed to ensure the richness of agent action sampling.

4.4. Heterogeneous multiworkflow collaborative scheduling algorithm based on D3QN

The D3QN algorithm is a fusion algorithm of double DQN algorithm and the dueling DQN algorithm, which inherits the double Q network structure of double DQN and the decoupling characteristics of target Q value action selection and calculation. At the same time, two subnetworks are designed in the double-Q network to learn the state value independent of action and the advantages related to action and state. Therefore, the D3QN algorithm is better than the DQN, double DQN and dueling DQN algorithms in terms of convergence and stability performance. The calculation formula of its target Q value is as follows:

$$y_i = R_j + \gamma Q'(\varnothing(S'_j), \arg\max_{a'} Q(\varnothing(S'_j), a, \theta), \theta') \quad (10)$$

In the formula, R_j is the reward for transferring to the next state S'_j after performing action A_j in the current state S_j . θ and θ' are the parameters of the Q network and the target Q network Q' respectively. γ is the discount factor, $\varnothing$ is the state map, and a are the specific actions. The value Q is defined as follows:

$$Q(S, a, \omega, \alpha, \beta) = V(S, \omega, \alpha) + \left(A(S, a, \omega, \beta) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(S, a', \omega, \beta) \right) \quad (11)$$

where V is the value function, A is the advantage function, ω is the public network layer parameter, α is the unique network parameter of the value function part, and β is the unique network parameter of the advantage function part. $\mathcal{A}$ is the action set, a' is the action in the action set, and $|\mathcal{A}|$ is the maximum number of actions. Combined with the aforementioned mechanisms and strategies, this paper applies the D3QN algorithm to the heterogeneous workflow scheduling process in the cloud computing environment, and proposes a heterogeneous multiworkflow collaborative scheduling algorithm based on the D3QN algorithm, as shown in Algorithm 4.

Algorithm 4: Heterogeneous multiworkflow collaborative scheduling algorithm based on D3QN

Input: Dynamic scheduling environment S , high-dimensional action set A , reward mechanism R
Output: Multiworkflow coscheduling strategy π^*

```

1 Initialize replay buffer  $D$ , Q&Target Q network with weights  $\theta$ 
2 observe  $S$  to initialize state  $s$ 
3 for  $i \in \text{max\_step}$  do
4    $a = \text{randomchoice}(A)$ 
5    $s' = \text{observe } S(a), r = R(s, a)$ 
6   store experience  $\langle s, a, r, s' \rangle$  in  $D$ 
7    $s = s'$ 
8 end
9 sample  $m$  transitions  $\langle s_-, a_-, r_-, s'_- \rangle$  from  $D$ 
10  $\text{loss} = \frac{1}{m} \sum_{i=1}^m (y_i - Q(\varnothing(S_i), A_i, \theta))^2$ 
11 train the Q network
12 share parameters from Q to Target Q
13 while not reach  $\text{max\_episode}$  do
14   for  $i \in \text{max\_step}$  do
15     if  $\text{random\_value} < \text{explore\_rate}$  then
16       select  $a$  softmax  $(Q(A))$ 
17     else
18        $a = \arg\max(Q(A))$ 
19     end
20     obtain  $r$  and  $s'$  from  $R$  and  $S$ 
21     store experience  $\langle s, a, r, s' \rangle$  in  $D$ 
22      $s = s'$ 
23   end
24   sample  $n$  transitions  $\langle s_-, a_-, r_-, s'_- \rangle$  from  $D$ 
25    $\text{loss} = \frac{1}{n} \sum_{i=1}^n (y_i - Q(\varnothing(S_i), A_i, \theta))^2$ 
26   train the Q network
27   share parameters from Q to Target Q
28   update  $D$ 
29 end

```

The multiworkflow collaborative scheduling algorithm selects the action sequence that maximizes the sum of the rewards in the dynamic scheduling environment and realizes the collaborative scheduling of a heterogeneous workflow under the high-dimensional objective. In this paper, the environmental features are composed of workflow characteristics, cloud server characteristics, virtual machine characteristics, and task characteristics, as shown in Table 1.

Based on the scheduling strategy and environment, this paper proposes a comprehensive reward mechanism combining long-term and short-term multielements to avoid the problems of sparse rewards and slow approximation. To adapt to the differences in the number and amount of computation of different workflows and maintain a stable learning environment, in this

Table 1
Environmental features.

Features	Description
$Stat(host_{ij})$	Status of the host
$Am(V_{ijk})$	Number of virtual machines available
$FinRate(W_i)$	Completion rate of workflow tasks
$Wait(W_i)$	Wait time of the workflow
$Wait(t_{ij})$	Wait time of the task
$Cp(t_{ij})$	Calculation amount of task
$AP(t_{ij})$	Number of parent tasks of the task
$AS(t_{ij})$	Number of child tasks of the task
$ASroot(t_{ij})$	Number of child task series
$FrWt(t_{ij})$	Wait time after the task status is free
L_i	Task quantity of a layer
Cp_i	Task calculation of a layer
$FreW_i$	Task free waiting time of a layer
Std_Cp_i	Standard deviation of Cp_i
Std_FreW_i	Standard deviation of $FreW_i$
tim_c	Wait time constraint

paper, we balance the total amount of computation and the total number of tasks in the reward. The upper limit of the continuity reward related to the total number of tasks is set in the rewards related to continuity, and the upper limit related to the total amount of calculation is set in other rewards, as follows:

$$R = \frac{Instant_r + Longtime_r + Main_r}{ACp(t_{ij})} \quad (12)$$

$Instant_r$ refers to the instant reward obtained after each scheduled action is executed. $Longtime_r$ is the reward that the time-step increases after the accumulation of multiple actions until a task is completed. $Main_r$ is the main line reward in the scheduling process. The magnitude of each reward can be adjusted according to the situation. Instant rewards are as follows:

$$Instant_r = sche_r + shift_r \quad (13)$$

Among them, $sche_r$ is defined as follows:

$$sche_r = (cpe_{tim} + cpe_{cost}) * Costcoe_i + ts_{k_{utr}} \quad (14)$$

When the task satisfies the continuity time constraint, the reward is the sum of the task computing time reward cpe_{tim} , computational cost reward cpe_{cost} , and the task waiting time reward $ts_{k_{utr}}$. The specific definitions of each part are as follows:

$$cpe_{tim} = \frac{host_{tmip} - \min(host_{mip})}{\max(host_{mip}) - \min(host_{mip})} * Cp(t_{ij}) \quad (15)$$

$$cpe_{cost} = \frac{Cp(t_{ij})}{host_{tmips}} * Cost(vm_{ij}) \quad (16)$$

$$Cost(vm_{ij}) = \frac{res(vm_{ij})}{res(host_i)} * (host_{statA} - host_{statS}) \quad (17)$$

$$ts_{k_{utr}} = \left(1 - \frac{FrWt(t_{ij})}{tim_c}\right) * \frac{ACp(t_{ij})}{All(t_{ij})} \quad (18)$$

$host_{tmip}$ is the computing power of the virtual machine selected by the current task, and cpe_{tim} reflects the advantages and disadvantages of computing power indicators in task scheduling. Therefore, the stronger the computing power is, the higher the reward. $Cost(vm_{ij})$ is the selected virtual machine service cost for the current task. $res(vm_{ij})$ and $res(host_i)$ are the total computing resources of the selected virtual machine and host, respectively. $host_{statA}$ is the cost coefficient under the condition of a full load of the host. $host_{statS}$ is the cost factor in the standby mode of the host. cpe_{cost} reflects the advantages and disadvantages of service cost indicators in task scheduling; that is, the lower the cost is, the higher the reward. $ts_{k_{utr}}$ corresponds to its continuity indicator

during task scheduling. The stronger the task continuity is, the greater the reward. The state switch reward is defined as follows:

$$shift_r = -ACp(t_{ij}) \quad (19)$$

The action reward for switching the host state is set to a negative value to avoid the agent choosing the above action without necessary conditions and improve the value of the host state switching action. In addition to the immediate reward, some rewards in the task scheduling process cannot be given immediately and need to be calculated after the scheduling clock changes. Therefore, it is necessary to use lag rewards to evaluate some indicators within a certain period of time in the scheduling process. The lag reward is defined as follows:

$$Longtime_r = -(workc_r + balanc_r) * time_{wi} \quad (20)$$

Among them, $workc_r$ is similar to cpe_{cost} , which represents the standby and sleep costs of all the hosts in the time window and is defined as follows:

$$workc_r = \begin{cases} host_{statS} * Costcoe_i & \text{if host is standby} \\ host_{statD} * Costcoe_i & \text{if host is dormant} \end{cases} \quad (21)$$

$host_{statS}$ is the standby cost coefficient of the host, and $host_{statD}$ is the dormant cost coefficient of the host. The definition of $balanc_r$ is consistent with $f_3(x)$. Therefore, the higher the fairness of the computing service, the greater the reward.

The mainline reward is the reward set by the scheduler to ensure that all tasks are scheduled to be completed. This is to prevent some tasks from being ignored when other rewards are too low or punishments are too strong. When the number of task executions reaches a fixed percentage of the total number of tasks, the mainline reward is generated, which is defined as follows:

$$Main_r = \begin{cases} \frac{\sum_{i=1}^M Fct(W_i)}{All(t_{ij})} & \text{if } \Gamma < All(t_{ij}) \\ \frac{\sum_{i=1}^M Fct(W_i)}{All(t_{ij})} + \frac{ACp(t_{ij})}{TIME} & \text{otherwise} \end{cases} \quad (22)$$

$$\Gamma = \sum_{i=1}^M Fct(W_i) \quad (23)$$

The above rewards are related to the constraints and objectives of the multiworkflow collaborative scheduling problem; additionally, they also affect the learning efficiency of reinforcement learning agents. Under the action of various types of rewards, the reinforcement learning agent is prompted to obtain better strategies and realize the collaborative scheduling of high-dimensional objectives of heterogeneous workflows in cloud computing.

5. Evaluation

5.1. Experimental environment and settings

In this paper, five types of heterogeneous workflow data are obtained from the existing workflow scheduling simulation data sources [39], namely CyberShake, Epigenomics, Inspiral, Montage, and Sipht. Various workflows have high structural and computational differences, which can verify the effectiveness of the method proposed in this paper, as shown in Table 2. In this paper, a simulation platform for heterogeneous workflow scheduling is built. The platform runs on a Windows 10 64-bit operating system and uses Python 3.8 and PyTorch 1.6.0 to train the scheduling model. The host hardware consists of an i710700F 2.9 GHz CPU, RTX2070S (8 GB) GPU and 32 GB RAM composition.

This paper sets up three groups of server clusters, each of which contains three different types of hosts, and the host can provide the corresponding virtual machine services. With reference to Alibaba Cloud server prices and host state switching

Table 2

Workflow data.

Workflow type	Number of tasks	Total computation
CyberShake	30	760.56
Epigenomics	24	71125.34
Inspirial	30	7473.52
Montage	25	227.30
Sipht	29	6065.87

Table 3

Computational cost of host in different states.

Cluster	Host	MIPs	$host_{stat A}$	$host_{stat S}$	$host_{stat D}$
A	c6.large	1000	0.418	0.2508	0.0836
	c6.xlarge	2000	0.808	0.4848	0.1616
	g6.xlarge	2500	1.028	0.6168	0.2056
B	c6.large	1000	0.577	0.3462	0.1154
	c6.xlarge	2000	1.126	0.6756	0.2252
	g6.xlarge	2500	1.486	0.8916	0.2972
C	c6.large	1000	0.706	0.4236	0.1412
	c6.xlarge	2000	1.376	0.8256	0.2752
	g6.xlarge	2500	1.790	1.0740	0.3580

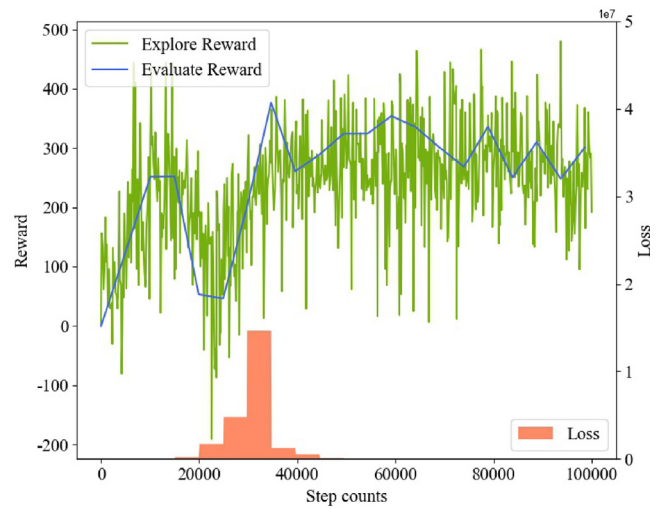
data [40,41], the host performance and operating costs in different states are simulated and set, as shown in Table 3. A, B, and C are computing clusters in different regions. The hosts in each cluster can provide different types of virtual machine services. $Cost_{coe_i}$ is 0.8, 1.0 and 1.2, respectively. When the host maintains cloud computing services, it can dynamically adjust the host state according to task requirements to balance the number of computing services. When the host is idle, it is in the standby or dormant state, and when the host resource is invoked, it is in the peak state. It takes 2.5 to 5 s to switch from dormancy to another state. For the convenience of calculation, the difference between the peak state cost coefficient of the host on the standby benchmark and the virtual machine resources is set as a linear relationship. In the scheduling process, in this paper, we ignore the data transmission factor of workflow tasks and only consider the timing characteristics of tasks within the workflow and its computational load.

5.2. Experimental results and analysis

5.2.1. Analysis of the heterogeneous workflow collaborative scheduling model

In this paper, the reasonable structure of the network model is determined in the experimental comparison of the increasing network depth to ensure sufficient information representation capabilities of the network. Two ten-layer deep neural target networks are set up, both of which have the same structure and consist of a five-layer state network, a five-layer advantage network and a five-layer value network, which share parameters over time. The training adopts a random strategy, the initial number of workflows is randomly set in the interval [2, 5], and each workflow type is randomly selected as one of five kinds of workflows. Through training, the reward and loss curves of deep reinforcement learning are obtained as shown in Fig. 6.

Fig. 6 shows that the agent can achieve good results and converge when the total number of exploration steps reaches approximately 100,000. Although the number of exploration rounds is less than the number of exploration steps, the number of workflow tasks in each round is large so that there are more learnable state–action–value pairs in each round. Therefore, the agent can learn general experience in a small number of rounds. To evaluate the effectiveness of the reward mechanism, under the continuous time constraint of 0.01 s, a new policy network is trained by scaling part of the reward, and the average value

**Fig. 6.** Model training curve.**Table 4**

Model results after adjusting the reward.

Adjustment	Make-span	Cost	Fairness	Continuity	Constraint
Baseline	6.714	0.0124	0.0105	0.0255	0.973
$cpe_{tim} * 2$	8.699	0.0161	0.0101	0.0073	0.987
$cpe_{tim}/2$	8.896	0.0164	0.0105	0.0075	0.985
$workc_r/2$	6.552	0.0122	0.0092	0.0199	0.982
$workc_r * 2$	6.588	0.0123	0.0101	0.0169	0.983
$balanc_r/100$	9.649	0.0177	0.0114	0.0073	0.988
$balanc_r * 100$	7.240	0.0134	0.0096	0.0168	0.984
$tsk_{wtr}/2$	7.186	0.0134	0.0100	0.0117	0.984
$tsk_{wtr} * 2$	6.602	0.0123	0.0093	0.0198	0.983

is obtained through multiple groups of training to acquire the performance of different models on multiple indicators under the same conditions, as shown in Table 4.

For the main reward parts, cpe_{tim} , $workc_r$, $balanc_r$, and tsk_{wtr} corresponding to makespan, cost, fairness and continuity, this paper scales each subpart based on the basic reward ratio. Table 4 shows that the subpart rewards proposed in this paper can increase or decrease the performance of the corresponding indicators after scaling. For example, increasing tsk_{wtr} , $workc_r$, and $balanc_r$ can optimize the corresponding indicators of continuity, cost and fairness. Decreasing cpe_{tim} and $balanc_r$ will lead to deterioration of the makespan and fairness metrics. However, there are also some noncorresponding situations, such as increasing cpe_{tim} to optimize the continuity index, and decreasing tsk_{wtr} to optimize the makespan index, which fully shows that the makespan index and the continuity index have mutual influence. Reducing $workc_r$ optimizes the overall performance, which also shows that the optimization of the overall performance is the result of the reasonable cooperation of multiple groups of rewards, and slight changes in the rewards will cause the agents to obtain different orientations.

From another dimension, the diversity of the solution sets means that our method can meet the diversity of scheduling needs. Generally, the requirements of the workflow are not always the same, and the preferences of the corresponding indicators are also quite different, which usually change with the business characteristics and time. For static or dynamic scheduling problems, traditional methods output a fixed or small number of solution sets and have limited coverage of the Pareto front. It is difficult to meet dynamic and diversified computing needs through the limited preferences of artificial design. However,

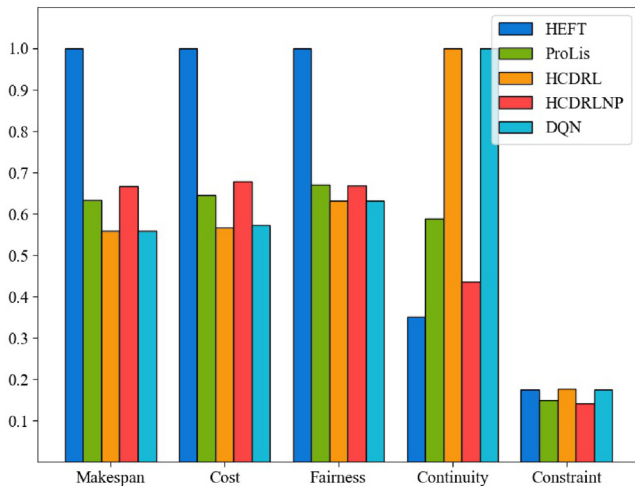


Fig. 7. Multi-index comparison results of different algorithms.

our method can adaptively follow the demand changes, and the output multiple solution sets can theoretically fully cover the Pareto front. Under the definition of demand characteristics, by adjusting our reward mechanism, we can achieve a highly flexible scheduling scheme, which is a great advantage of the artificial intelligence method to quickly follow the demand changes.

5.2.2. Comparative analysis of high-dimensional objective collaborative scheduling

This paper quantifies the effect of the proposed method by comparing it with the HEFT [42] algorithm, the ProLis [43] algorithm without preference scheduling (HCDRLNP) and the DQN algorithm. The HEFT algorithm is a typical static scheduling algorithm in the workflow scheduling algorithm, which schedules all tasks to the virtual machine that can complete it earliest. The ProLis algorithm first selects the task with the longest task waiting time for priority scheduling. On this basis, it prefers the optimal cost. If the conditions are not met, the task that can be completed earliest is given priority. The HCDRLNP algorithm is a scheduling algorithm that removes the equilibrium and priority task loading strategies of the HCDRL algorithm and is used to verify the effectiveness of the task loading algorithm. The DQN algorithm is a scheduling algorithm in which the HCDRL algorithm replaces the D3QN with the DQN training, and the corresponding network layers and parameters are consistent with the D3QN.

In this paper, a random control group for scheduling heterogeneous workflows is set up, and the above algorithm is used to schedule multiple groups of heterogeneous workflows randomly generated under the same conditions as the training process. Each objective takes the maximum value as the base, and the value is scaled in the [0, 1] interval. The continuity constraint is shown as ten times the proportion of tasks that violate the continuity time constraint. The comparison results are shown in Fig. 7.

As seen in Fig. 7, the algorithm proposed in this paper is considerably better than other algorithms in terms of makespan, cost and fairness. In terms of makespan, the HCDRL and DQN algorithms are consistent, outperforming the HEFT, ProLis, and HCDRLNP algorithms by 44.1%, 11.8%, and 16.3% respectively. The superior performance of the makespan indicator shows that the HCDRL algorithm can outperform the comparison algorithm in this indicator even without the host state control action. Because the host state control will actually affect the computing time, when the server faces a sudden increase in computing demand, it will take time to switch the state and activate more service resources. Therefore, the advanced scheduling scheme based on

prediction is a feasible research direction for future work, which can maximize the optimization of server state control decisions. The cost index is 43.3%, 12.1%, 16.6% and 1.1% better than the HEFT, ProLis, HCDRLNP and DQN algorithms, respectively. Theoretically, there is an approximate linear relationship between the cost index and the time index, but HCDRL is better than the time index in cost control. This is due to the addition of the host state control action, which avoids cost waste under resource redundancy. At the same time, the advantages of the HCDRL over the DQN also illustrate the excellent performance of the D3QN algorithm applied in this paper. In terms of fairness, the HCDRL is also consistent with the DQN algorithm, outperforming the HEFT, ProLis and HCDRLNP algorithms by 36.8%, 5.8% and 5.3%, respectively. This shows that the algorithm proposed in this paper can pay more attention to fair performance. The random task loading mode of HEFT and ProLis leads to the decline of their service fair performance. Due to the limitations of the workflow structure, some workflows may obtain more or fewer resources in the random mode. As a result, the seemingly fair random mode is affected by the characteristics of workflow, and it is easy to produce the phenomenon of unfair resource allocation according to the definition of workflow structure. The disadvantage of the HCDRLNP algorithm also shows the effectiveness of the equilibrium strategy.

In terms of continuity indicators, the algorithm proposed in this paper has a slight disadvantage with the DQN algorithm. Because the algorithm in this paper involves control of the host state, it has a certain impact on the service response speed. The reward ratio in the current strategy is biased toward other indicators, and the decline of the continuity indicator is exchanged for other indicators. Although the performance of the algorithm on the continuity index is worse than that of other algorithms, it can still ensure that more than 98% of the tasks follow the continuity constraint of 0.01 s, and the performance of the constraints is not much different from other algorithms. This shows that the continuity constraint compliance rate can be optimized under the application of the task priority loading strategy proposed in this paper. Furthermore, in Fig. 7, we can see that without the application of the task loading strategy, the algorithm has poor support for makespan, cost and fairness indicators, which shows that the flexible application of the task loading strategy can play a role in multiple objectives. It can be seen that the HCDRL algorithm has advantages in the three indicators of makespan, cost, and fairness, and can better follow the continuity constraints.

On this basis, this paper selects different load conditions to compare the above algorithms. The number of workflows is set in [2, 5], the number of different workflows in the number interval is used for multiple experiments, and the average value is obtained. The comparison results of multiple indicators are shown in Fig. 8. According to the result, the HCDRL algorithm is superior to the HEFT, ProLis, HCDRLNP and DQN algorithms in terms of makespan, cost and fairness, which shows the stability of the HCDRL algorithm under different load conditions. In terms of the continuity index, the HCDRL algorithm has advantages in the four load cases where the lower limit of the number of workflows is 2, and when the lower limit is 3, the continuity index attenuates more than other algorithms. This shows that HCDRL has learned a better scheduling strategy under low load conditions and can achieve high-dimensional optimal scheduling of the four objectives. With sufficient training, our algorithm can be closer to the Pareto frontier than other algorithms, and with the support of multiple groups of solutions, our method can achieve the dual advantages of solution quality and richness. It also shows that the learning effect of HCDRL is not ideal under the load situation with a large number of workflows, and there is still room for optimization. This is also in line with the general

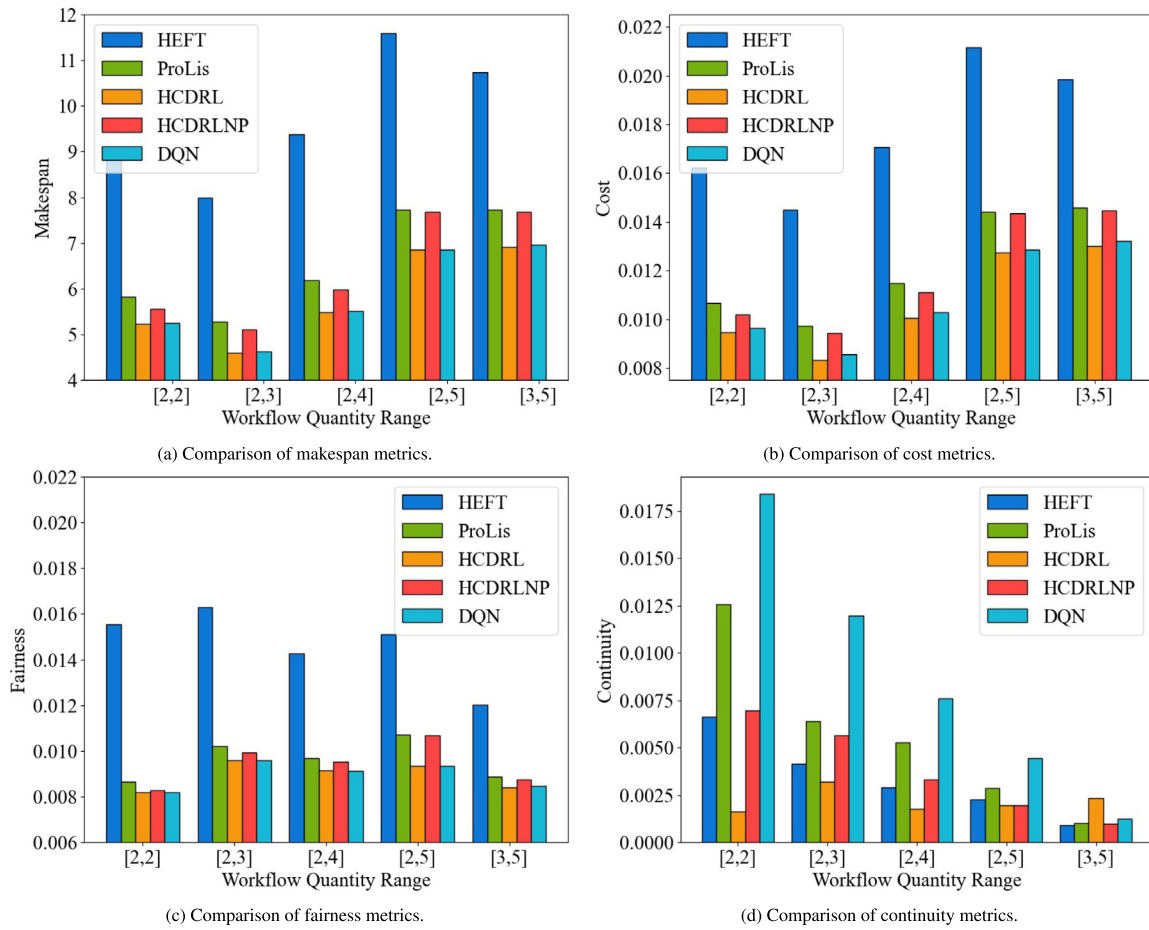


Fig. 8. Comparison results of multiple indicators under different load conditions.

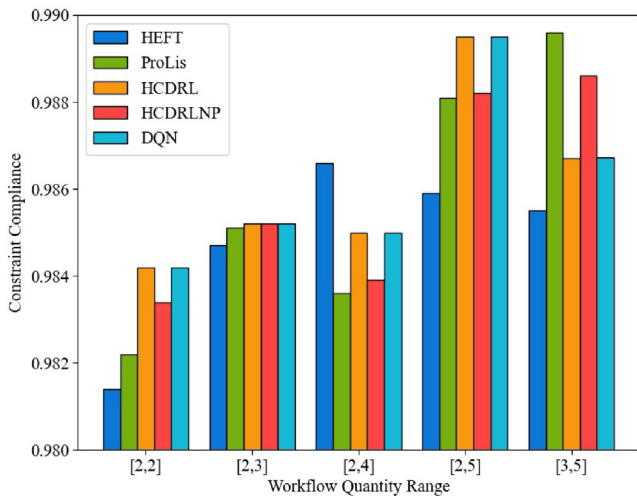


Fig. 9. Constraint compliance rate comparison results for different load conditions.

law of agent learning, that is, the more complex the environment, the more difficult the learning, and the less ideal the performance. This shows that the deep reinforcement learning solution for large-scale heterogeneous workflow scheduling still needs to be further optimized.

Correspondingly, the constraint compliance rate under different load conditions is shown in Fig. 9. The HCDRL algorithm has

advantages when the number of workflows is in the interval [2,2], [2,3], [2,5]. Under other conditions, HCDRL can also achieve an intermediate level, which can guarantee a 98% constraint compliance rate. This depends on our reward mechanism. By setting a penalty term related to the continuity constraint, agents can be effectively prevented from making decisions that break the constraint. In the interval [3,5], the continuity index of the HCDRL algorithm in Fig. 8 is the worst. However, in Fig. 9, the continuity index compliance rate of the HCDRL algorithm surpasses that of the HEFT algorithm and reaches a level consistent with the DQN algorithm. This shows that the HCDRL algorithm can still achieve a good constraint compliance effect when the continuity index performance is average, and the optimization and coordination of the objectives can be achieved by flexibly configuring the high-dimensional scheduling strategy. In the [2,5] interval, the continuous index of HCDRLNP in Fig. 8 is optimal, and the constraint compliance index in the corresponding interval in Fig. 9 deteriorates, indicating that the algorithm objective coordination ability is relatively poor without applying the task loading strategy.

In summary, HCDRL can optimize high-dimensional objectives, such as makespan, cost, fairness, and continuity of heterogeneous workflow execution in cloud computing environments under the premise of satisfying continuity constraints.

6. Conclusion

With the increasing demand for cloud computing services, the coupling of massive computing services complicates the process of collaborative scheduling of heterogeneous workflows, and

high-dimensional objective optimization problems such as optimization and adaptation of cloud computing services and efficient demand response have also attracted widespread attention. In this paper, a collaborative scheduling method based on the D3QN is designed for the high-dimensional objective scheduling optimization problem of heterogeneous workflows in cloud computing under the continuity constraint. On the basis of effectively perceiving the state of time-series tasks in the scheduling process, a time-step adaptive scheduling mechanism is adopted, and a scheduling-switching hybrid action, equilibrium, priority, preference high-dimensional strategy and multifactor fusion reward mechanism are designed. By intelligently controlling the task scheduling scheme and the computing service environment, this paper innovatively realizes the collaborative optimization of high-dimensional objectives on both the supply and demand sides, such as the makespan, cost, fairness and continuity of heterogeneous workflows in a cloud computing environment. The effectiveness of the proposed method is verified by experimental simulations using heterogeneous workflows. The experimental results show that under the constraint of 98% task continuity, the method in this paper can optimize the cost by 43.3% compared with the HEFT algorithm, and improve the time performance by 11.8% compared with the ProLis algorithm. Including the task loading strategy can optimize the makespan performance by 16.6%, the cost by 16.3%, and improve the fairness index by 5.3%.

Although this paper proposes a solution for collaborative scheduling of high-dimensional objectives of heterogeneous workflows, there are still some directions worthy of research in the future. For larger-scale workflow scheduling problems, the combination of higher-dimensional knowledge-inspired scheduling strategies and hierarchical deep reinforcement learning methods should be considered to improve the applicability of the method. For the highly dynamic cloud computing service environment, we can consider observing multiscale and multimodal spatiotemporal information and design a precise advance scheduling method supported by multidimensional information to achieve the effect of a refined and rapid configuration of cloud computing service resources.

CRedit authorship contribution statement

Genxin Chen: Methodology, Software, Writing – original draft. **Jin Qi:** Data curation, Project administration. **Ying Sun:** Visualization, Investigation. **Xiaoxuan Hu:** Software, Validation. **Zhenjiang Dong:** Supervision. **Yanfei Sun:** Conceptualization, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgments

This work is supported by the General Program of the National Natural Science Foundation of China (62172235) and the National Key Research and Development Program of China (2021YFB310-1100).

References

- [1] S. Wang, Z. Ding, C. Jiang, Elastic scheduling for microservice applications in clouds, *IEEE Trans. Parallel Distrib. Syst.* 32 (1) (2021) 98–115.
- [2] R. Xu, Y. Wang, H. Luo, F. Wang, Y. Xie, X. Liu, Y. Yang, A sufficient and necessary temporal violation handling point selection strategy in cloud workflow, *Future Gen. Comput. Syst. Int. J. Esci.* 86 (2018) 464–479.
- [3] A. Song, W.-N. Chen, X. Luo, Z.-H. Zhan, J. Zhang, Scheduling workflows with composite tasks: A nested particle swarm optimization approach, *IEEE Trans. Serv. Comput.* 15 (2) (2022) 1074–1088.
- [4] W.-N. Chen, J. Zhang, An ant colony optimization approach to a grid workflow scheduling problem with various qos requirements, *IEEE Trans. Syst. Man Cybern. C (Applications and Reviews)* 39 (1) (2009) 29–43.
- [5] F. Li, M.G. Seok, W. Cai, A new double rank-based multi-workflow scheduling with multi-objective optimization in cloud environments, in: 2021 IEEE International Parallel and Distributed Processing Symposium WORKSHOPS, IPDPSW, 2021, pp. 36–45.
- [6] Y.-H. Jia, W.-N. Chen, H. Yuan, T. Gu, H. Zhang, Y. Gao, J. Zhang, An intelligent cloud workflow scheduling system with time estimation and adaptive ant colony optimization, *IEEE Trans. Syst. Man Cybern. Syst.* 51 (1) (2021) 634–649.
- [7] L. Ye, Y. Xia, L. Yang, C. Yan, Shws: Stochastic hybrid workflows dynamic scheduling in cloud container services, *IEEE Trans. Autom. Sci. Eng.* (2021) 1–17.
- [8] J. Liu, J. Ren, W. Dai, D. Zhang, P. Zhou, Y. Zhang, G. Min, N. Najjari, Online multi-workflow scheduling under uncertain task execution time in iaas clouds, *IEEE Trans. Cloud Comput.* 9 (3) (2021) 1180–1194.
- [9] J.K. Konjaang, L. Xu, Cost optimised heuristic algorithm (coha) for scientific workflow scheduling in iaas cloud environment, in: 2020 IEEE 6TH INT Conference on Big Data Security on Cloud (BIGDATASECURITY) / 6TH IEEE INT Conference on High Performance and Smart Computing, (HPSC) / 5TH IEEE INT Conference on Intelligent Data and Security (IDS), 2020, pp. 162–168.
- [10] E. Cadorel, H. Coullon, J.-M. Menaud, Online multi-user workflow scheduling algorithm for fairness and energy optimization, in: L. Lefevre, C. Varela, G. Pallis, A. Toosi, O. Rana, R. Buyya (Eds.), 2020 20TH IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID 2020), 2020, pp. 569–578.
- [11] R.F. d. Silva, W. Chen, G. Juve, K. Vahi, E. Deelman, Community resources for enabling research in distributed scientific workflows, in: 2014 IEEE 10th International Conference on e-Science, vol. 1, 2014, pp. 177–184.
- [12] X. Tang, Reliability-aware cost-efficient scientific workflows scheduling strategy on multi-cloud systems, *IEEE Trans. Cloud Comput.* (2021) 1.
- [13] J. Zhou, J. Sun, M. Zhang, Y. Ma, Dependable scheduling for real-time workflows on cyber-physical cloud systems, *IEEE Trans. Ind. Inform.* 17 (11) (2021) 7820–7829.
- [14] Q. Wu, M. Zhou, J. Wen, Endpoint communication contention-aware cloud workflow scheduling, *IEEE Trans. Autom. Sci. Eng.* 19 (2) (2022) 1137–1150.
- [15] W. Chen, E. Deelman, Fault tolerant clustering in scientific workflows, in: 2012 IEEE Eighth World Congress on Services, 2012, pp. 9–16.
- [16] H. Chen, X. Zhu, G. Liu, W. Pedrycz, Uncertainty-aware online scheduling for real-time workflows in cloud service environment, *IEEE Trans. Serv. Comput.* 14 (4) (2021) 1167–1178.
- [17] M. Mao, M. Humphrey, Auto-scaling to minimize cost and meet application deadlines in cloud workflows, in: SC '11: Proceedings of, in: 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, 2011, pp. 1–12.
- [18] J. Du, L. Zhao, J. Feng, X. Chu, Computation offloading and resource allocation in mixed fog/cloud computing systems with min-max fairness guarantee, *IEEE Trans. Commun.* 66 (4) (2018) 1594–1608.
- [19] W. Wu, J. Wang, K. Lu, W. Qi, F. Shan, J. Luo, Providing service continuity in clouds under power outage, *IEEE Trans. Serv. Comput.* 13 (5) (2020) 930–943.
- [20] M. Shahid, Z. Ashraf, M. Alam, F. Ahmad, M. Imran, A multi-objective workflow allocation strategy in iaas cloud environment, in: 2021 International Conference on Computing, Communication, and Intelligent Systems, ICCIS, 2021, pp. 308–313.
- [21] Rupali N. Mangla, A critical review of workflow scheduling algorithms in cloud computing environment, in: 2021 Fourth International Conference on Computational Intelligence and Communication Technologies (CCICT), 2021, pp. 355–361.
- [22] T.-P. Pham, T. Fahringer, Evolutionary multi-objective workflow scheduling for volatile resources in the cloud, *IEEE Trans. Cloud Comput.* (2020) 1.
- [23] B. Hu, Z. Cao, M. Zhou, Energy-minimized scheduling of real-time parallel workflows on heterogeneous distributed computing systems, *IEEE Trans. Serv. Comput.* (2021) 1.

- [24] Z. Wen, S. Garg, G.S. Aujla, K. Alwasel, D. Puthal, S. Dustdar, A.Y. Zomaya, R. Ranjan, Running industrial workflow applications in a software-defined multicloud environment using green energy aware scheduling algorithm, *IEEE Trans. Ind. Inform.* 17 (8) (2021) 5645–5656.
- [25] H. Li, B. Wang, Y. Yuan, M. Zhou, Y. Fan, Y. Xia, Scoring and dynamic hierarchy-based nsga-ii for multiobjective workflow scheduling in the cloud, *IEEE Trans. Autom. Sci. Eng.* 19 (2) (2022) 982–993.
- [26] M. Niu, B. Cheng, Y. Feng, J. Chen, Gmta: A geo-aware multi-agent task allocation approach for scientific workflows in container-based cloud, *IEEE Trans. Netw. Serv. Manag.* 17 (3) (2020) 1568–1581.
- [27] S. Baer, J. Bakakeu, R. Meyes, T. Meisen, Multi-agent reinforcement learning for job shop scheduling in flexible manufacturing systems, in: 2019 Second International Conference on Artificial Intelligence for Industries, AI4I, 2019, pp. 22–25.
- [28] X. Tang, W. Shi, F. Wu, Interconnection network energy-aware workflow scheduling algorithm on heterogeneous systems, *IEEE Trans. Ind. Inform.* 16 (12) (2020) 7637–7645.
- [29] X. Tang, K. Li, G. Liao, K. Fang, F. Wu, A stochastic scheduling algorithm for precedence constrained tasks on grid, *Future Gen. Comput. Syst. Int. J. Esci.* 27 (8) (2011) 1083–1091.
- [30] H.R. Faragardi, M.R. Saleh Sedghpour, S. Fazlihamadi, T. Fahringer, N. Rasouli, Grp-heft: A budget-constrained resource provisioning scheme for workflow scheduling in iaas clouds, *IEEE Trans. Parallel Distrib. Syst.* 31 (6) (2020) 1239–1254.
- [31] K. Deng, K. Ren, M. Zhu, J. Song, A data and task co-scheduling algorithm for scientific cloud workflows, *IEEE Trans. Cloud Comput.* 8 (2) (2020) 349–362.
- [32] M. Dickinson, S. Debroy, P. Calyam, S. Valluripally, Y. Zhang, R.B. Antequera, T. Joshi, T. White, D. Xu, Multi-cloud performance and security driven federated workflow management, *IEEE Trans. Cloud Comput.* 9 (1) (2021) 240–257.
- [33] G.-Y. Zhu, C. Ding, W.-B. Zhang, Optimal foraging algorithm that incorporates fuzzy relative entropy for solving many-objective permutation flow shop scheduling problems, *IEEE Trans. Fuzzy Syst.* 28 (11) (2020) 2738–2746.
- [34] Y. Feng, L. Feng, Y. Hou, K.C. Tan, S. Kwong, Emt-remo: Evolutionary multitasking for high-dimensional multi-objective optimization via random embedding, in: 2021 IEEE Congress on Evolutionary Computation, CEC, 2021, pp. 1672–1679.
- [35] J. Huang, X. Huang, Y. Ma, Y. Liu, On a high-dimensional objective genetic algorithm and its nonlinear dynamic properties, *Commun. Nonlinear Sci. Numer. Simul.* 16 (9) (2011) 3825–3834.
- [36] H. Djigal, J. Feng, J. Lu, J. Ge, Iptps: An efficient algorithm for scientific workflow scheduling in heterogeneous computing systems, *IEEE Trans. Parallel Distrib. Syst.* 32 (5) (2021) 1057–1071.
- [37] L. Ran, X. Shi, M. Shang, Slas-aware online task scheduling based on deep reinforcement learning method in cloud environment, in: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), 2019, pp. 1518–1525.
- [38] A. Paya, D.C. Marinescu, Energy-aware load balancing and application scaling for the cloud ecosystem, *IEEE Trans. Cloud Comput.* 5 (1) (2017) 15–27.
- [39] W. Chen, E. Deelman, Workflowsim: A toolkit for simulating scientific workflows in distributed environments, in: 2012 IEEE 8th International Conference on E-Science, 2012, pp. 1–8.
- [40] X. Fan, W.-D. Weber, L.A. Barroso, Power provisioning for a warehouse-sized computer, in: ISCA'07: 34TH Annual International Symposium on Computer Architecture, Conference Proceedings, 2007, pp. 13–23.
- [41] Y. Agarwal, S. Hodges, R. Chandra, J. Scott, P. Bahl, R. Gupta, Somniloquy: Augmenting network interfaces to reduce pc energy usage, in: Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation, NSDI'09, USENIX Association, USA, 2009, pp. 365–380.
- [42] H. Topcuoglu, S. Hariri, M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Trans. Parallel Distrib. Syst.* 13 (3) (2002) 260–274.
- [43] Q. Wu, F. Ishikawa, Q. Zhu, Y. Xia, J. Wen, Deadline-constrained cost optimization approaches for workflow scheduling in clouds, *IEEE Trans. Parallel Distrib. Syst.* 28 (12) (2017) 3401–3412.



Genxin Chen received the B.S. degree in automation from Jiangxi University of Science and Technology, Ganzhou, China, in 2019. He is currently working toward the Ph.D. degree in Information Acquisition and Control with the School of Automation, Nanjing University of Posts and Telecommunications, Nanjing, China. His current research interests include deep reinforcement learning, cloud computing and distributed computing.



Industrial Internet, and Energy Internet.

Jin Qi received the B.S. degree in computer science and technology from Liaocheng University, Liaocheng, China, in 2005. He received the M.S. degree in computer application from Jiangnan University, Wuxi, China, in 2008. He received the Ph. D. degrees in information network from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2012. He is currently an Associate Professor at School of Internet of Things in Nanjing University of Posts and Telecommunications since December 2012. His main research interests are in the areas of Intelligent Computing,



Ying Sun received the Ph. D degree from Nanjing University of Posts and Telecommunications. She is currently with the Nanjing University of Posts and Telecommunications. Her research interests include software engineering and machine learning.



Xiaoxuan Hu received her BS and Ph.D. in School of Automation, Nanjing University of Posts and Telecommunications, Nanjing, China, in 2014 and 2021, respectively. She is currently a Lecture in the School of Internet of Things, Nanjing University of Posts and Telecommunications. Her research interests mainly focus on industrial Internet, energy Internet, cloud computing, big data processing and smart grid.



Zhenjiang Dong received the Ph.D. degree from Shanghai Jiao Tong University, Shanghai, China. He is currently a full professor at School of Computer Science in Nanjing University of Posts and Telecommunications. His current research interests are mainly in the area of computer vision, Internet of Vehicles, high-precision positioning, natural language processing and knowledge graph.



Yanfei Sun received the Ph.D. degree in communication and information systems from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2006. He is currently a full professor at School of Internet of Things in Nanjing University of Posts and Telecommunications. His current research interests are mainly in the area of future networks, industrial Internet, energy Internet, big data management and analysis, and intelligent optimization and control.